

CEENs: Causality-enforced evolutionary networks for solving time-dependent partial differential equations

Jeahan Jung¹, Heechang Kim¹, Hyomin Shin, Minseok Choi^{*}

Department of Mathematics, Pohang University of Science and Technology (POSTECH), Pohang 37673, Republic of Korea

ARTICLE INFO

Keywords:

Physics-informed neural network
Long-time integration
Partial differential equation
Predictive modeling
Nonlinear dynamics

ABSTRACT

Despite the growing popularity of physics-informed neural networks (PINNs), their applicability in the long-time integration of partial differential equations (PDEs) remains constrained. We argue that this problem stems from the lack of consideration of temporal causality in the original PINN formulation, resulting in a bias towards satisfying governing equations at later times before learning the initial condition and hence leading to erroneous solutions. To this end, we propose a novel method that seamlessly integrates temporal causality into the training process. Drawing inspiration from classical numerical methods where the temporal causality is reflected, we divide the time domain into nonoverlapping subintervals, assign a unique neural network to each subinterval, and construct a loss function founded on the integral form of PDEs within these subintervals. The proposed networks undergo sequential training, beginning with the initial time step. Our method demonstrates significant improvement in accuracy for long-time simulations of various PDE problems where the original PINN method fails while it requires less computational cost and memory compared to the PINN method. A parallelization algorithm is provided to further enhance the computational efficiency, showing a significant speedup for solving time-dependent PDEs.

1. Introduction

The tremendous growth of machine learning technology has led to explosive advancement in various fields such as natural language processing, data mining, and computer vision. In particular, physics-informed neural networks (PINNs) [1–5] have been proposed and become a new trend of solving partial differential equations (PDEs) using machine learning. The effectiveness of PINNs has been demonstrated in a variety of PDE problems including the forward and inverse problems [1,6–9], and uncertainty quantification [8–17]. The use of neural networks for PDE has also been proposed three decades ago [18,19], but use of large-scale neural networks was computationally prohibitive at the time. Advances in computer hardware such as graphic processing units (GPUs) and recent Python libraries such as Tensorflow [20], Pytorch [21], and JAX [22] provide tools for machine learning-based PDE solver.

Despite the notable achievements of PINNs in solving PDEs, there remains a significant challenge in integrating time-dependent PDEs given initial conditions [23–29]. Time-dependent PDEs frequently arise in various science and engineering models; therefore, it is important to understand the underlying pathologies of PINNs in handling such situations and enhance their efficacy in addressing the challenges. The literature has highlighted that a major obstacle of PINNs in solving time-dependent PDEs is the lack of consideration of temporal causality in the training process. This means that PINNs tend to be biased towards satisfying

^{*} Corresponding author.

E-mail addresses: obok13@postech.ac.kr (J. Jung), heechangkim@postech.ac.kr (H. Kim), zhainl@postech.ac.kr (H. Shin), mchoi@postech.ac.kr (M. Choi).

¹ Authors contributed equally.

governing equations at later times before even learning the initial condition, leading to an inaccurate solution while having low training loss. To address this pathology, a number of variants of PINNs have been proposed to incorporate temporal causality during training. In [24,27,28], sequential learning strategies of PINN have been proposed to learn the solution from early time steps. The authors in [25,30–35] employ recurrent neural networks, which are known to effectively handle time series data, to capture temporal causality. In [26,36,37], the authors train operators that predict a solution after a certain period of time from the current state. The iterative application of these operators, starting from the initial condition, is used to determine a solution at a specific time. The methods proposed in [23,24,38,39] divide the time domain into short intervals and train a solution in each interval. The authors in [1,29] assign a neural network at each time step and train them sequentially. The loss functions of the assigned neural networks are defined using the Runge–Kutta method, widely used method of integrating ordinary differential equations (ODEs).

Taking cues from classical numerical methods that naturally respect temporal causality, we propose a method involving time-domain decomposition and enforcing sequential learning from the initial time. Our method divides the time domain into non-overlapping subintervals, assigning a unique neural network to each subinterval, and constructs a loss function based on the integral form of PDEs within these subintervals. Sequential training is enforced, beginning with the initial condition and proceeding through each subsequent subinterval. Although similar to the discrete PINN method based on the Runge–Kutta method in [1], our approach offers key distinctions and advantages, including the construction of a loss function using a linear multi-step method that allows for parallelization by avoiding the need for intermediate step calculations. In particular, we use the trapezoidal rule for the loss function, known for its second-order accuracy in terms of the time step. In addition we introduce a parallelization algorithm that can significantly accelerate the training for the long-time integration of PDEs. Our main contributions can be summarized as:

- We propose a novel method to address the problem of temporal causality for time-dependent PDEs by dividing the time domain into subintervals, each equipped with a neural network, and enforcing a sequential training that initiates from the very initial time step and proceeds through each subsequent subinterval.
- We provide a parallelization algorithm designed to expedite the training process, providing a substantial speedup and enhanced computational efficiency.
- We demonstrate the effectiveness of our model for learning long-time integration of PDEs where the original PINN method fails.

The rest of the paper is organized as follows. In Section 2, we introduce the original formulation of PINN proposed in [1] and explain why it fails to reflect temporal causality in the training process. To address this limitation, we propose a new model in Section 3 that effectively incorporates temporal causality and describe its training process. In addition, we elaborate on practical considerations to improve the efficiency of our proposed method and provide the algorithm. To evaluate the effectiveness of the proposed method, Section 4 presents state-of-the-art results in a set of challenging time-dependent PDE problems where the original PINN method fails to solve. In Section 5, we conclude the paper with a brief summary, some limitations, and future research directions.

2. Physics-informed neural networks (PINNs)

In this section, we briefly introduce an overview of PINNs [1] for solving PDE problems. We consider the time-dependent PDEs of the form:

$$u_t + \mathcal{N}[u] = 0, t \in [0, T], x \in \Omega \quad (1)$$

$$u(0, x) = g(x), x \in \Omega \quad (2)$$

$$\mathcal{B}[u] = 0, t \in [0, T], x \in \partial\Omega \quad (3)$$

where $\mathcal{N}$ and $\mathcal{B}$ are differential and boundary operators, respectively, $\Omega \subset \mathbb{R}^d$ is a spatial domain with a boundary $\partial\Omega$, and u is the solution of the equation. We assume that the initial condition g is a smooth function. Note that in the case where g is a nonsmooth function, there are methods to effectively approximate such nonsmooth functions [40,41]. In the PINN formulation, the solution $u(t, x)$ is represented by a neural network model $U(t, x; \theta)$ with its trainable parameters θ . The loss function is designed to ensure that the model output satisfies the initial, boundary condition, and the governing equation, i.e.,

$$\mathcal{L}^{\text{PINN}}(\theta) = \mathcal{L}_{ic}^{\text{PINN}}(\theta) + \mathcal{L}_{bc}^{\text{PINN}}(\theta) + \mathcal{L}_{res}^{\text{PINN}}(\theta)$$

where

$$\begin{aligned} \mathcal{L}_{ic}^{\text{PINN}}(\theta) &= \frac{1}{N_{ic}} \sum_{j=1}^{N_{ic}} |U(0, x_{ic}^j; \theta) - g(x_{ic}^j)|^2, \\ \mathcal{L}_{bc}^{\text{PINN}}(\theta) &= \frac{1}{N_{bc}} \sum_{j=1}^{N_{bc}} |\mathcal{B}[U](t_{bc}^j, x_{bc}^j; \theta)|^2, \\ \mathcal{L}_{res}^{\text{PINN}}(\theta) &= \frac{1}{N_{res}} \sum_{j=1}^{N_{res}} |U_t(t_{res}^j, x_{res}^j; \theta) + \mathcal{N}[U](t_{res}^j, x_{res}^j; \theta)|^2. \end{aligned} \quad (4)$$

$\{x_{ic}^j\}_{j=1}^{N_{ic}}$, $\{t_{bc}^j, x_{bc}^j\}_{j=1}^{N_{bc}}$, and $\{t_{res}^j, x_{res}^j\}_{j=1}^{N_{res}}$ denote sampled points to approximate L^2 norm on Ω , $[0, T] \times \partial\Omega$, and $[0, T] \times \Omega$, respectively. The ultimate goal of PINNs is to determine θ that minimizes the loss function $\mathcal{L}^{\text{PINN}}(\theta)$, which is typically achieved by stochastic gradient-based optimization algorithms. Note that the temporal and spatial gradients of the model output included in (4) can be efficiently calculated by automatic differentiation.

However, this formulation of PINN may not incorporate temporal causality in the training process. As demonstrated in [27], PINNs trained via stochastic gradient descent are implicitly biased towards approximating the solution to PDEs at later time steps first, even prior to learning the initial conditions, hence violating temporal causality. Consequently, PINNs often fail to solve time-dependent PDEs, despite having low training loss.

To provide a further clarification, we consider the following one-dimensional convection equation.

$$\begin{aligned} u_t + 15u_x &= 0, \quad \text{for } t \in [0, 5], x \in [0, 2\pi], \\ u(0, x) &= \sin x, \\ u(t, 0) &= u(t, 2\pi). \end{aligned}$$

We employ a fully-connected network $U(t, x; \theta)$ with a hyperbolic tangent activation function to represent the solution $u(t, x)$. The structure of the network includes 2 hidden layers, and 820 neurons in each hidden layer. We enforce periodic boundary conditions as hard constraints by utilizing a suitable Fourier features embedding of the input as introduced in [42]. It ensures that the periodic boundary condition is directly integrated into the model's architecture, enhancing the model's ability to accurately represent and solve problems with inherent periodic structures. We use a uniform grid of size 2001×101 in the spatio-temporal domain $[0, 5] \times [0, 2\pi]$ for creating initial, boundary, and residual points. The PINN model is then trained using Adam optimizer for 100,000 iterations. Fig. 1(a) displays the PINN model's performance and shows that the PINN model fails to provide an accurate solution for this example. Specifically, the model predicts the solution to be near zero after $t = 1$. Fig. 1(b) depicts loss convergence of the initial condition loss $\mathcal{L}_{ic}^{\text{PINN}}$ and the PDE residual loss $\mathcal{L}_{res}^{\text{PINN}}$ during training. Notably, the residual loss function decreases during the initial few thousand training epochs and then shows negligible changes throughout the remaining training period. This behavior indicates that the predicted solution is biased towards an inappropriate local minimum. The blue solid line and green dashed line in Fig. 1(c) represent the residual loss of PINN at $t = 0$ and 1, respectively, defined by

$$\tilde{\mathcal{L}}_{res}^{\text{PINN}}(t, \theta) = \frac{1}{N_x} \sum_{j=1}^{N_x} |U_t(t, x^j; \theta) + \mathcal{N}[U](t, x^j; \theta)|^2 \quad (5)$$

where $\mathcal{N}[U] = 15U_x$ and x^j 's are randomly sampled points on Ω . As $\tilde{\mathcal{L}}_{res}^{\text{PINN}}(t, \theta)$ approximates the norm of residual term at time t , we can use it to assess whether the approximate solution at time t satisfies the governing equation. A crucial observation is that $\tilde{\mathcal{L}}_{res}^{\text{PINN}}(1, \theta)$ decreases rapidly in the initial training epochs, while $\tilde{\mathcal{L}}_{res}^{\text{PINN}}(0, \theta)$ remains large. This reveals a potential violation of temporal causality, wherein the solution at a later time step ($t = 1$) is being optimized before adequate training has taken place at an earlier time step ($t = 0$). Such a phenomenon can lead to a series of predictions at later time steps that, while adhering to the governing equations, are biased towards an undesired local minimum, in this case zero, which does not represent a physically viable solution. This observation underscores the importance of adhering to temporal causality in the optimization process; it is critical to prioritize the minimization of residual loss at earlier time steps before moving onto later ones, ensuring a more accurate and physically coherent solution.

Remark. The average of $\tilde{\mathcal{L}}_{res}^{\text{PINN}}(t, \theta)$ evaluated on sampled points $t = t_0, t_1, \dots, t_N \in [0, T]$ is equivalent to $\mathcal{L}_{res}^{\text{PINN}}(\theta)$ by the following.

$$\begin{aligned} \frac{1}{N} \sum_{n=0}^N \tilde{\mathcal{L}}_{res}^{\text{PINN}}(t_n, \theta) &\approx \frac{1}{T} \int_0^T \tilde{\mathcal{L}}_{res}^{\text{PINN}}(t, \theta) dt \approx \frac{1}{T} \frac{1}{|\Omega|} \int_0^T \int_{\Omega} |U_t(t, x; \theta) + \mathcal{N}[U](t, x; \theta)|^2 dx dt \\ &\approx \frac{1}{N_r} \sum_{j=1}^{N_r} |U_t(t_r^j, x_r^j; \theta) + \mathcal{N}[U](t_r^j, x_r^j; \theta)|^2 = \mathcal{L}_{res}^{\text{PINN}}(\theta) \end{aligned}$$

Unfortunately, this issue is not uncommon when applying PINNs to time-dependent PDEs, as evidenced by a growing body of recent research [23–29]. In these cases, the accuracy of the solution deteriorate as the time variable increases, leading to increasingly inaccurate predictions as time progresses. This becomes particularly problematic in systems where the target solution exhibits a substantial dependency on the initial conditions, such as those found in chaotic systems. Therefore, it is crucial to develop methods that adequately account for temporal causality. By doing so, we can ensure consistently accurate predictions over the entire time domain.

3. Causality-enforced evolutionary networks (CEENs)

In this section, we propose causality-enforced evolutionary networks to enforce the sequential learning and its parallelization to enhance the computational efficiency, followed by a detailed analysis of the proposed method.

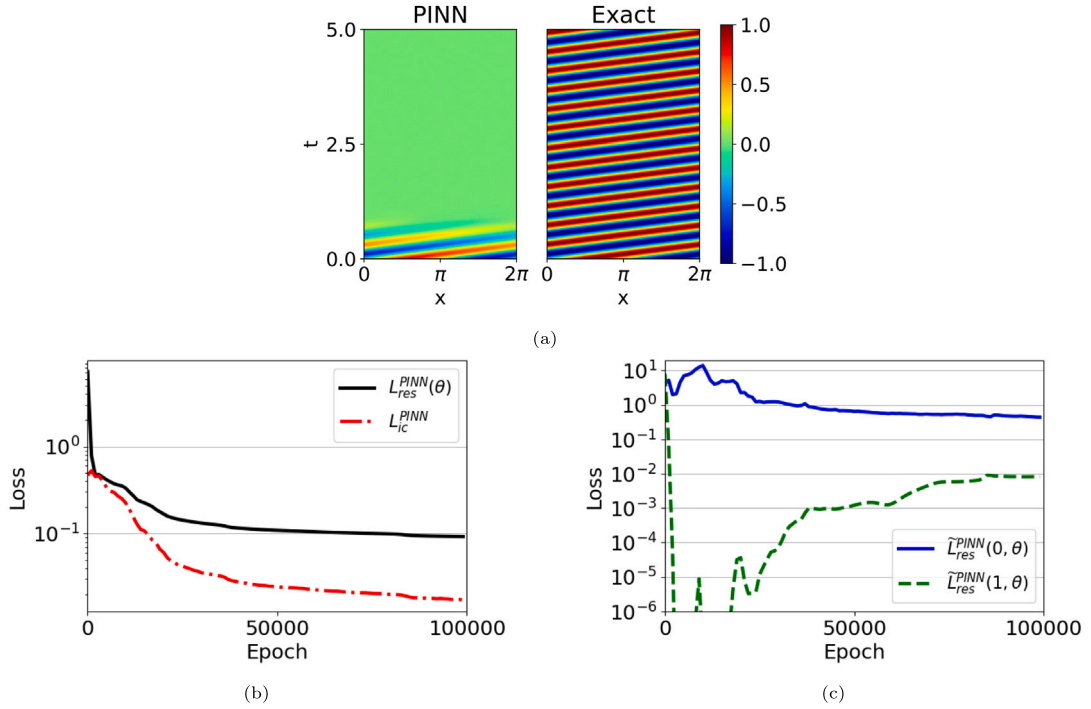


Fig. 1. PINN performance for the one-dimensional convection equation. (a) The prediction of a trained conventional PINN versus reference solution. The PINN model provides an inaccurate prediction. (b) Initial condition loss of PINN and residual loss of PINN during training. (c) Residual losses of PINN at $t = 0$ and 1 . The residual loss of PINN at $t = 1$ remains low, while that of PINN at $t = 0$ remains large. This suggests a violation of temporal causality in the learning process.

3.1. Methodology of CEENs

We present a method for solving time-dependent PDEs that incorporates temporal causality into its formulation. We call this method the causality-enforced evolutionary network (CEEN). As highlighted in many studies, integrating the concept of temporal causality into model construction when solving time-dependent PDEs is critical. Our proposed model addresses this requirement by assigning distinct neural networks at each time step and sequentially training them, beginning from the initial time step.

More specifically, we divide the temporal domain $[0, T]$ into N equidistant subintervals $0 = t_0 < t_1 < \dots < t_N = T$. At each time step $t = t_n$, we assign a neural network model $U^n(x; \theta^n)$ with its unique parameter θ^n to approximate $u(t_n, x)$. The loss function used to train the designated neural network $U^n(x; \theta^n)$ is derived from the governing physical law within the subinterval $[t_{n-1}, t_n]$. To provide a detailed account, consider the integral form of the governing equation (1) over a subinterval $[t_{n-1}, t_n]$:

$$u(t_n, x) = u(t_{n-1}, x) - \int_{t_{n-1}}^{t_n} \mathcal{N}[u](t, x) dt. \quad (6)$$

Linear multi-step methods in the classical numerical methods depend on the choice of the approximation of the integral in (6). For better illustration, from now on, we assume to use the trapezoidal rule to approximate the integral:

$$\int_{t_{n-1}}^{t_n} \mathcal{N}[u](t, x) dt = \frac{1}{2} h (\mathcal{N}[u](t_{n-1}, x) + \mathcal{N}[u](t_n, x)) + \frac{\mathcal{N}[u]_t(t_*, x)}{12} h^3 \quad (7)$$

where $t_* \in [t_{n-1}, t_n]$ and $h = T/N$. Combining Eq. (7) with the initial and boundary conditions ((2) and (3)), we derive the relationships between the neural networks for $n = 1, 2, \dots, N$ as follows:

$$\begin{aligned} U^0(x; \theta^0) &= g(x) \quad \text{for } x \in \Omega, \\ B[U^n](x; \theta^n) &= 0 \quad \text{for } x \in \partial\Omega. \\ U^n(x; \theta^n) &= U^{n-1}(x; \theta^{n-1}) - \frac{1}{2} h (\mathcal{N}[U^{n-1}](x; \theta^{n-1}) + \mathcal{N}[U^n](x; \theta^n)) \quad \text{for } x \in \Omega, \end{aligned} \quad (8)$$

The loss function of CEEN is then designed to ensure that the designated networks satisfy (8) at each time step $t_n, n = 1, 2, \dots, N$:

$$\mathcal{L}^{\text{CEEN}}(\theta) = \mathcal{L}_{ic}^{\text{CEEN}}(\theta) + \mathcal{L}_{bc}^{\text{CEEN}}(\theta) + \mathcal{L}_{res}^{\text{CEEN}}(\theta)$$

where the components of the loss function are defined as:

$$\begin{aligned}
 \mathcal{L}_{ic}^{\text{CEEN}}(\theta) &= \frac{1}{N_x} \sum_{j=1}^{N_x} |U^0(x^j; \theta^0) - g(x^j)|^2, \\
 \mathcal{L}_{bc}^{\text{CEEN}}(\theta) &= \frac{1}{N} \sum_{n=1}^N \frac{1}{N_b} \sum_{j=1}^{N_b} B[U^n](x_{bc}^j, \theta^n)^2 =: \frac{1}{N} \sum_{n=1}^N \tilde{\mathcal{L}}_{bc}^{\text{CEEN}}(t_n, \theta^n), \\
 \mathcal{L}_{res}^{\text{CEEN}}(\theta) &= \frac{1}{N} \sum_{n=1}^N \frac{1}{N_x} \sum_{j=1}^{N_x} \left| \frac{U^n(x^j; \theta^n) - U^{n-1}(x^j; \theta^{n-1})}{h} + \frac{1}{2} (\mathcal{N}[U^{n-1}](x^j; \theta^{n-1}) + \mathcal{N}[U^n](x^j; \theta^n)) \right|^2 \\
 &=: \frac{1}{N} \sum_{n=1}^N \tilde{\mathcal{L}}_{res}^{\text{CEEN}}(t_n, \theta^n, \theta^{n-1}).
 \end{aligned} \tag{9}$$

$\{x^j\}_{j=1}^{N_x}$ and $\{x_{bc}^j\}_{j=1}^{N_b}$ denote sampled points to approximate L^2 norm on Ω and $\partial\Omega$, respectively. The loss functions $\tilde{\mathcal{L}}_{bc}^{\text{CEEN}}(t, \theta^n)$ and $\tilde{\mathcal{L}}_{res}^{\text{CEEN}}(t, \theta^n, \theta^{n-1})$ in (9) represent the boundary and residual losses of CEEN at each time step $t = t_n, n = 1, 2, \dots, N$.

In contrast to PINN, our method does not seek to minimize the total loss $\mathcal{L}^{\text{CEEN}}(\theta)$ over the entire time domain. Instead, to account for temporal causality, we prioritize sequential network training, beginning from the initial time step. Accordingly, we split the loss function $\mathcal{L}^{\text{CEEN}}(\theta)$ into:

$$\mathcal{L}^{\text{CEEN}}(\theta) = \mathcal{L}_0^{\text{CEEN}}(\theta^0) + \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n^{\text{CEEN}}(\theta^n, \theta^{n-1})$$

where

$$\begin{aligned}
 \mathcal{L}_0^{\text{CEEN}}(\theta^0) &= \mathcal{L}_{ic}^{\text{CEEN}}(\theta) \\
 \mathcal{L}_n^{\text{CEEN}}(\theta^n, \theta^{n-1}) &= \tilde{\mathcal{L}}_{bc}^{\text{CEEN}}(t_n, \theta^n) + \tilde{\mathcal{L}}_{res}^{\text{CEEN}}(t_n, \theta^n, \theta^{n-1}).
 \end{aligned}$$

We can enforce the sequential network training by systematically minimizing the loss function $\mathcal{L}_n^{\text{CEEN}}$ in ascending order of $n = 0, 1, \dots, N$. More specifically, we initiate by training θ^0 to decrease $\mathcal{L}_0^{\text{CEEN}}(\theta^0)$ until it drops below a specified tolerance, $\epsilon > 0$. Once we identify the optimal parameter $\bar{\theta}^0$ satisfying the condition $\mathcal{L}_0^{\text{CEEN}}(\bar{\theta}^0) < \epsilon$, we stop training θ^0 . We move on to train θ^1 to minimize $\mathcal{L}_1^{\text{CEEN}}(\theta^1, \bar{\theta}^0)$ until we identify $\bar{\theta}^1$ that satisfies the condition $\mathcal{L}_1^{\text{CEEN}}(\bar{\theta}^1, \bar{\theta}^0) < \epsilon$. Note that the value of $\bar{\theta}^0$ remains constant while θ^1 is being trained, implying that the past state U^0 is unaffected by the current state U^1 . After identifying the optimal parameter $\bar{\theta}^1$, we proceed to find $\bar{\theta}^2$ such that $\mathcal{L}_2^{\text{CEEN}}(\bar{\theta}^2, \bar{\theta}^1) < \epsilon$. We continue this iterative process until we obtain the final trained parameter $\bar{\theta}^N$ satisfying $\mathcal{L}_N^{\text{CEEN}}(\bar{\theta}^N, \bar{\theta}^{N-1}) < \epsilon$. This training procedure sequentially reduces the residual loss at each time step, unlike PINN, thereby ensuring that the solution respects the temporal causality.

3.2. Enhancing CEENs with parallelization

The total number of neural networks required for training is $N + 1 = T/h + 1$, which can be quite large for a long time T or small time step size h . While training each neural network over a short time interval is generally quicker than training the original PINN over the whole time domain, the process could still demand considerable computational resources. To mitigate these computational demands, we propose a parallelization algorithm. This approach enables us to simultaneously train M successive neural networks instead of adhering to the traditional method of training a single neural network for each time step. To be concrete, in the beginning of the training process, we train the networks $U^0, U^1, \dots, U^{M-1}$ instead of solely training U^0 . In this case, the loss function $\mathcal{L}_{0:M-1}^{\text{CEEN}}$ of their parameters $\theta^0, \theta^1, \dots, \theta^{M-1}$ is the sum of $\mathcal{L}_0^{\text{CEEN}}, \mathcal{L}_1^{\text{CEEN}}, \dots, \mathcal{L}_{M-1}^{\text{CEEN}}$, i.e.,

$$\mathcal{L}_{0:M-1}^{\text{CEEN}}(\theta^0, \theta^1, \dots, \theta^{M-1}) = \mathcal{L}_0^{\text{CEEN}}(\theta^0) + \sum_{k=1}^{M-1} \mathcal{L}_k^{\text{CEEN}}(\theta^k, \theta^{k-1}).$$

In order to use gradient-based optimization methods, we require its gradient:

$$\nabla_{(\theta^0, \theta^1, \dots, \theta^{M-1})} \mathcal{L}_{0:M-1}^{\text{CEEN}} = \nabla_{\theta^0} \mathcal{L}_0^{\text{CEEN}} + \sum_{k=1}^{M-1} \nabla_{(\theta^k, \theta^{k-1})} \mathcal{L}_k^{\text{CEEN}},$$

which can be calculated in parallel by concurrently computing each gradient term across multiple GPU cores. As a result, the computational cost of training multiple networks becomes comparable to that of training a single network, with a decrease in the total number of required epochs, consequently reducing the computational cost.

It is important to note that we must account for temporal causality during parallel training. For example, should $\mathcal{L}_1^{\text{CEEN}}(\theta^1, \bar{\theta}^0) < \epsilon$ be achieved before $\mathcal{L}_0^{\text{CEEN}}(\bar{\theta}^0) < \epsilon$, it would be incorrect to stop the training of θ^1 before the completion of θ^0 training, as this would violate temporal causality. Therefore, we initially optimize the loss $\mathcal{L}_{0:M-1}^{\text{CEEN}}$ until $\mathcal{L}_0^{\text{CEEN}}(\bar{\theta}^0)$ is sufficiently reduced. Once we find $\bar{\theta}^0$ that makes $\mathcal{L}_0^{\text{CEEN}}(\bar{\theta}^0) < \epsilon$, we stop training for the parameter θ^0 and focus on minimizing the subsequent loss function $\mathcal{L}_{1:M-1}^{\text{CEEN}}$:

$$\mathcal{L}_{1:M-1}^{\text{CEEN}}(\theta^1, \theta^2, \dots, \theta^{M-1}) = \mathcal{L}_1^{\text{CEEN}}(\theta^1, \bar{\theta}^0) + \sum_{k=2}^{M-1} \mathcal{L}_k^{\text{CEEN}}(\theta^k, \theta^{k-1}).$$

Upon finding $\bar{\theta}^1$ satisfying $\mathcal{L}_1^{\text{CEEN}}(\bar{\theta}^1, \bar{\theta}^0) < \epsilon$, we stop the training of θ^1 , and then proceed to minimize the loss function $\mathcal{L}_{2:M-1}^{\text{CEEN}}$, which comprises the remaining loss functions. This iterative process continues until we find $\bar{\theta}^{M-1}$ satisfying $\mathcal{L}_{M-1}^{\text{CEEN}}(\bar{\theta}^{M-1}, \bar{\theta}^{M-2}) < \epsilon$. The training of the next set of neural networks of size M then follows the same pattern.

Remark. As described in the above parallelization strategy, it is crucial to maintain a sequential order even when training M networks simultaneously. This constraint introduces additional overhead, explaining why the computational cost does not scale linearly as M increases as demonstrated in Section 4. A large M may be constrained by the GPU resources. Consequently, the optimal selection of M must be determined heuristically, considering the available GPU capacity.

3.3. Analysis on CEENs

In this subsection, we discuss a detailed analysis of the proposed method such as the error bound of the solution at each subinterval, the training of initial condition, memory efficiency, and comparison with the conventional numerical methods.

The choice of time step size and tolerance The time step size h and the tolerance ϵ are important hyperparameters for the accuracy of the prediction, with the potential to introduce significant errors at each time step if chosen incorrectly. Specifically, a large time step size h can result in considerable approximation errors, which are proportional to h^2 due to the trapezoidal rule. Conversely, setting the tolerance ϵ too high may lead to substantial errors in the predicted solution stemming from insufficient training. Note that the exact solution over a subinterval $[t_{n-1}, t_n]$ should satisfy the following equation, which can be derived from (6) and (7),

$$u(t_n, x) - u(t_{n-1}, x) + \frac{1}{2}h\mathcal{N}[u](t_{n-1}, x) + \mathcal{N}[u](t_n, x) + \frac{\mathcal{N}[u]_{tt}(t_*, \cdot)}{12}h^3 = 0.$$

Since $U^n(x; \theta^n)$ with parameters θ^n represents the solution at t_n satisfying Eq. (8), we can obtain the following L^2 error:

$$\begin{aligned} & \|U^n - U^{n-1} + \frac{1}{2}h(\mathcal{N}[U^{n-1}] + \mathcal{N}[U^n]) + \frac{\mathcal{N}[u]_{tt}(t_*, \cdot)}{12}h^3\|_{L^2(\Omega)} \\ & \leq \|U^n - U^{n-1} + \frac{1}{2}h(\mathcal{N}[U^{n-1}] + \mathcal{N}[U^n])\|_{L^2(\Omega)} + \frac{1}{12}h^3\|\mathcal{N}[u]_{tt}(t_*, \cdot)\|_{L^2(\Omega)} \\ & \approx \left[\frac{|\Omega|}{N_x} \sum_{i=1}^{N_x} \left| U^n(x^i; \theta^n) - U^{n-1}(x^i; \theta^{n-1}) + \frac{1}{2}h(\mathcal{N}[U^{n-1}](x^i; \theta^{n-1}) + \mathcal{N}[U^n](x^i; \theta^n)) \right|^2 \right]^{1/2} \\ & \quad + \frac{1}{12}h^3\|\mathcal{N}[u]_{tt}(t_*, \cdot)\|_{L^2(\Omega)} \\ & \leq |\Omega|^{1/2}(\mathcal{L}_n^{\text{CEEN}})^{1/2}h + \frac{1}{12}h^3\|\mathcal{N}[u]_{tt}(t_*, \cdot)\|_{L^2(\Omega)} \\ & \leq |\Omega|^{1/2}\epsilon^{1/2}h + \frac{1}{12}h^3\|\mathcal{N}[u]_{tt}(t_*, \cdot)\|_{L^2(\Omega)} =: e_1(\epsilon, h) + e_2(h) \end{aligned} \quad (10)$$

where $e_1(\epsilon, h) := |\Omega|^{1/2}\epsilon^{1/2}h$ and $e_2(h) := \frac{1}{12}h^3\|\mathcal{N}[u]_{tt}(t_*, \cdot)\|_{L^2(\Omega)}$. Note that the error over a subinterval $[t_{n-1}, t_n]$ consists of two terms, $e_1(\epsilon, h)$ and $e_2(h)$. $e_1(\epsilon, h)$ represents the training loss, dependent on ϵ and h , while $e_2(h)$ accounts for the consistency error of the trapezoidal rule, dependent on h . A large value of ϵ can lead to a significant $e_1(\epsilon, h)$ error at each time step. Conversely, choosing a small ϵ may not be advantageous either. Despite a small ϵ leading to a lower value of e_1 , it could entail an increased training cost to bring $\mathcal{L}_n^{\text{CEEN}}$ below ϵ at each time step. Given that the error e_2 is inevitably incurred by the trapezoidal approximation, choosing an ϵ that significantly reduces e_1 below e_2 is unnecessary. Ideally, ϵ should be selected to satisfy $e_1(\epsilon, h) \approx e_2(h)$. However, this principle might not always be practical as the value of $\|\mathcal{N}[u]_{tt}(t_*, \cdot)\|_{L^2(\Omega)}$ is generally unknown. Regardless, this principle can still offer valuable guidance for choosing ϵ as the time step size h varies.

The error bound, characterized by two components e_1 and e_2 , reveals the order of the accuracy with respect to the time step size and tolerance. If ϵ is sufficiently small such that e_1 's impact on total error is minor, then the error is dominated by e_2 . This suggests a second-order accuracy with respect to h , consistent with expectations from the Trapezoidal rule. Conversely, when h is reduced to a point where e_2 's effect on the error is minimal, e_1 becomes the main contributor to the error, indicating a half-order accuracy in terms of ϵ . The validation of this relationship will be examined in Section 4.3, providing empirical support for these theoretical assertions.

The training of initial condition One crucial aspect of the method that requires careful consideration is the training of the initial condition (2). Note that a small value of $\mathcal{L}_0^{\text{CEEN}}$ does not automatically ensure an accurate approximation of $\mathcal{N}[g]$ by $\mathcal{N}[U^0]$, which appears in $\mathcal{L}_1^{\text{CEEN}}$. This is primarily because the differential operator $\mathcal{N}$ is generally not a continuous linear operator. In other words, the following inequality with some constant $C > 0$ does not hold in general:

$$\|\mathcal{N}[U^0] - \mathcal{N}[g]\|_{L^2(\Omega)} \leq C\|U^0 - g\|_{L^2(\Omega)}$$

This can lead to significant errors when calculating $\mathcal{L}_1^{\text{CEEN}}$ as $\mathcal{N}[U^0]$ may deviate from $\mathcal{N}[g]$. The initial condition loss and residual loss at $t = 0$ in Fig. 1(b, c) effectively capture this phenomenon. To address this issue, we propose to utilize the modified loss of $\mathcal{L}_0^{\text{CEEN}}$ incorporating the derivatives:

$$\mathcal{L}_0^{\text{CEEN}}(\theta) = \sum_{k=0}^K \frac{1}{N_x} \sum_{j=1}^{N_x} |D^k U^0(x^j; \theta^0) - D^k g(x^j)|^2, \quad (11)$$

where $D^k u$ denotes the list of all k th order partial derivatives of u and $\mathcal{N}$ is assumed to be the Lipschitz function of u , Du , D^2u , ..., $D^K u$. This adjustment ensures that a small value of $\mathcal{L}_0^{\text{CEEN}}$ corresponds to a more accurate approximation of $\mathcal{N}[g]$ by $\mathcal{N}[U^0]$. For implementing this formulation, we assume that precise knowledge of the initial conditions or data on their derivatives is available.

Memory efficiency The memory efficiency of CEENs is superior compared to PINNs, attributed to several reasons. Firstly, unlike PINNs that aim to model solutions across the entire spatio-temporal domain, CEENs focus on modeling solutions within a short temporal domain, thus leveraging smaller networks. This implies that the number of parameters to be updated during the training of CEEN is significantly lower compared to PINNs. Secondly, while PINNs' training utilizes points spanning the entire spatio-temporal domain, CEENs employ fewer training points, confined to a short temporal domain. Additionally, CEENs avoid the usage of automatic differentiation in the time direction, resulting in a further decrease in memory usage. Therefore, CEEN demands considerably less memory for gradient computation in updating parameters compared to PINN. While PINN occupies the GPU with the update of numerous parameters, CEEN processes parameter updates swiftly, even within constrained GPU environments. This memory efficiency potentially makes large-scale simulations of complex problems feasible, which might prove challenging or even unachievable when using PINNs.

Maximum training epoch There may be instances when $\mathcal{L}_n^{\text{CEEN}}$ cannot be minimized to the desired extent. To this end, we introduce the maximum number of training epochs, denoted by E , to prevent excessive iterations so that the training of θ^n is finished even if $\mathcal{L}_n^{\text{CEEN}} < \epsilon$ is not achieved. If the loss is not sufficiently reduced over many consecutive time steps, errors can accumulate and negatively affect the accuracy of the overall solution. In this case, we should consider a change in the architecture of the neural network as it may not be suitable to represent the solution. For example, for modeling high-frequency solutions, we can consider using a Fourier feature network [5,43].

Comparison with classical numerical methods The proposed method is similar to classical numerical methods for solving time-dependent PDEs in the sense that it sequentially computes the solution at each time step. However, CEEN presents several advantages over these classical methods. Firstly, CEEN can precisely compute the spatial derivatives of the solution. In contrast, conventional mesh-based numerical methods often require the dense placement of spatial points to achieve the desired accuracy in spatial derivatives. CEEN, by leveraging a neural network for each time step, facilitates the exact computation of derivatives via automatic differentiation, thereby eliminating approximation errors common in traditional methods. Secondly, CEEN offers flexibility in the positioning of spatial points. Traditional numerical methods might necessitate specific arrangements of spatial points to ensure accurate computation of derivatives, a restriction CEEN does not impose. This adaptability proves advantageous, particularly in scenarios where data is only available at certain locations within the spatial domain. Thirdly, after training on specific spatial points, the solution at any arbitrary point in CEEN can be instantly obtained with a single forward pass through the trained network. This feature significantly enhances the efficiency and applicability of the method, providing immediate solutions at any desired location. Finally, leveraging automatic differentiation for the precise calculation of spatial derivatives enables us to overcome traditional constraints on time step and spatial grid sizes mandated by classical methods for stability, such as the Courant–Friedrichs–Lewy (CFL) condition. This advantage allows for the selection of time step sizes independent of grid dimensions. However, unlike classical methods that have established convergence analyses, such as the Lax Equivalence Theorem, an equivalent convergence analysis for PINNs, addressing aspects like generalization error, is not yet available. This area, requiring a more profound understanding of PINNs, remains beyond the scope of this paper but represents a significant avenue for future research.

A schematic diagram representing CEEN is shown in Fig. 2. Algorithm 1 summarizes the proposed method outlined above.

Algorithm 1 Causality-enforced evolutionary network (CEEN)

Input: Maximum epoch E , Number of networks computed in parallel M , tolerance ϵ , learning rate η .

```

1: Initialize  $\theta = (\theta^0, \theta^1, \dots, \theta^N)$  and  $m = 0$ 
2: repeat
3:   for  $n = m, m + 1, \dots, m + M - 1$  do
4:      $\mathcal{L}_{n:m+M-1}^{\text{CEEN}}(\theta^n, \theta^{n+1}, \dots, \theta^{m+M-1}) = \mathcal{L}_n^{\text{CEEN}}(\theta^n, \overline{\theta^{n-1}}) + \sum_{k=n+1}^{m+M-1} \mathcal{L}_k^{\text{CEEN}}(\theta^k, \theta^{k-1})$ 
5:     while  $\mathcal{L}_{n:m+M-1}^{\text{CEEN}}(\theta^n, \overline{\theta^{n-1}}) < \epsilon$  do
6:        $(\theta^n, \theta^{n+1}, \dots, \theta^{m+M-1}) = (\theta^n, \theta^{n+1}, \dots, \theta^{m+M-1}) - \eta \nabla_{(\theta^n, \theta^{n+1}, \dots, \theta^{m+M-1})} \mathcal{L}_{n:m+M-1}^{\text{CEEN}}$ 
7:       if  $\mathcal{L}_n^{\text{CEEN}} < \epsilon$  is not achieved for  $E$  epochs then
8:         break
9:       Save the parameter  $\theta^n$  as  $\overline{\theta^n}$ 
10:     $m = m + M$ 
11: until All network parameters are trained
Output: Trained parameters  $\overline{\theta^0}, \overline{\theta^1}, \dots, \overline{\theta^N}$ 

```

4. Numerical results

We demonstrate the effectiveness of our proposed CEEN algorithm for solving a range of time-dependent PDEs over long time intervals. Specifically, we apply our method to the Korteweg–De Vries (KdV), Schrödinger, convection, and Allen–Cahn equations. The performance of our method is then compared to that of the PINN. As summarized in Table 1, our algorithm (CEEN) consistently

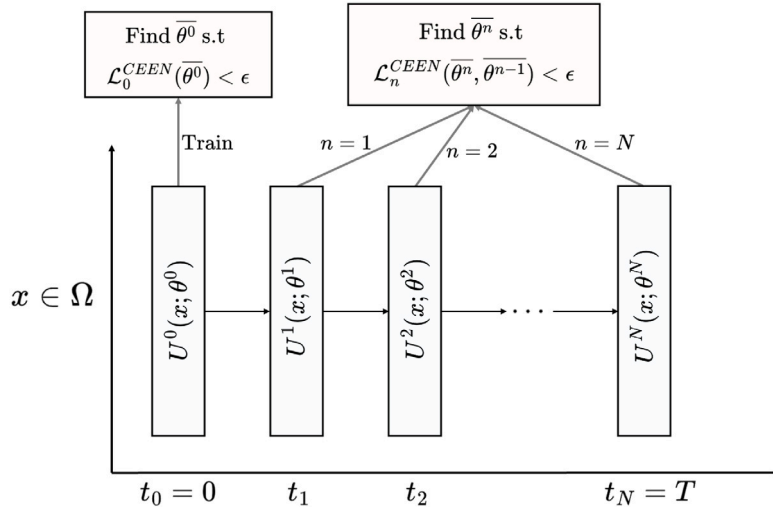


Fig. 2. Schematic of CEEN. The temporal domain is discretized into subintervals, with a neural network assigned to each time step. The solution is obtained by sequentially training each network, beginning from the initial time. This approach reflects temporal causality in the training process.

Table 1
Relative errors of CEEN and PINN at the final time $t = 5$ for four PDE problems.

Example	Relative error	
	CEEN	PINN
KdV	5.314e-3	1.237e-0
Schrödinger	4.793e-3	2.346e-1
Convection	5.465e-3	1.000e-0
Allen-Cahn	4.222e-2	1.394e-0

delivers higher accuracy at the final time $t = 5$ across all these problems compared to PINN. This underscores the superior performance of our proposed method.

We compute reference solutions utilizing the py-pde package [44], with a time step size of $1e-7$. The prediction of both CEEN and PINN are compared against these reference solutions. Across all problems, parameters are initialized via the Glorot normal scheme, the activation function is set as hyperbolic tangent, and the maximum training epochs are capped at $E = 100,000$. For each problem, the value of K in (11), which is employed to learn the initial condition, is set to be the highest order of the derivative present in the associated differential operator $\mathcal{N}$. All networks are trained through the Adam optimizer with $\beta = (0.999, 0.999)$, a configuration providing stable learning [45]. For all problems except Allen-Cahn problem, we enforce periodic boundary conditions as hard constraints by utilizing an appropriate Fourier features embedding of the input as outlined in [42].

4.1. KdV equation

The first example aims to demonstrate the proposed method's capability to address PDEs characterized by high order derivatives and dispersive nonlinearity. For this purpose, we consider the KdV equation, a mathematical model representing shallow water surface waves and ionic acoustic waves in plasmas. The KdV equation takes the form

$$\begin{aligned}
 u_t + uu_x + 0.0025u_{xxx} &= 0, \quad \text{for } t \in [0, 5], x \in [-1, 1] \\
 u(0, x) &= \cos \pi x, \\
 u(t, -1) &= u(t, 1), \\
 u_x(t, -1) &= u_x(t, 1).
 \end{aligned} \tag{12}$$

To apply our method, we discretize the time domain with a time step size $h = 1e-2$. A fully-connected network with two hidden layers, each containing 16 neurons, is utilized to model the solution $U^i(x; \theta^i)$ at each time step t_i . We select $N_x = 151$ equidistant points on the spatial domain and set the tolerance to $\epsilon = 1e-4$. The values of K and M used are 3 and 5, respectively.

For a fair comparison of our method with PINN, we design the architectures for PINN so that both PINN and CEEN have nearly the same number of parameters. Specifically, our method uses 501 neural networks, each consisting of 465 parameters, resulting in a total of 232,965 parameters. The PINN model, on the other hand, is constructed with a fully-connected network with two hidden layers each containing 480 neurons. In total, the PINN model has 237,121 parameters.

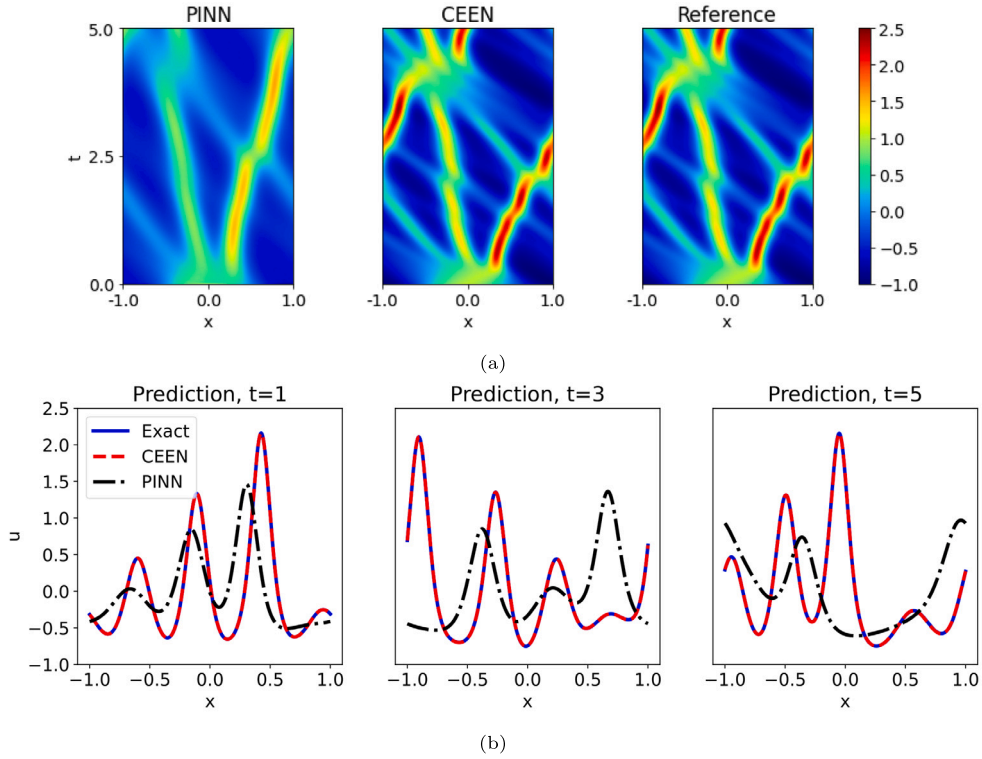


Fig. 3. *KdV equation*: Comparison of the prediction of CEEN and PINN against the reference solution. (a) displays the prediction across the entire spatio-temporal domain. (b) displays it at three different times: $t = 1, 3, 5$. CEEN agrees well with the reference in the entire domain while PINN exhibits a less accurate approximation of the solution even at an earlier time $t = 1$.

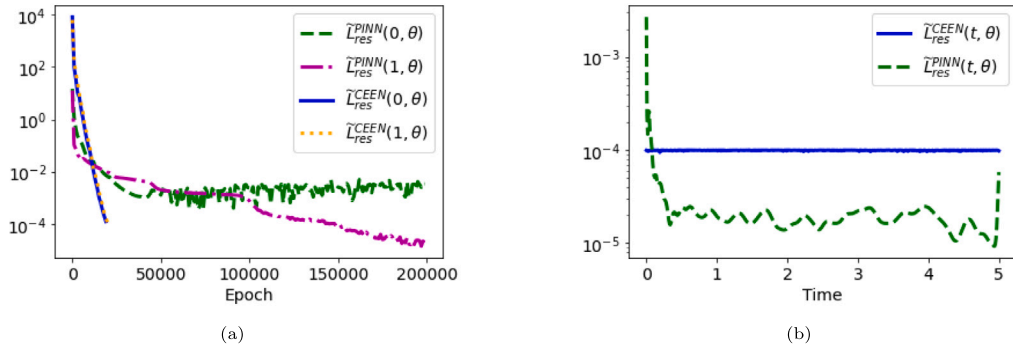


Fig. 4. Residual losses of CEEN and PINN for the KdV equation (12). (a) Epoch vs residual loss. (b) Time vs residual loss.

As shown in Fig. 3, our proposed method accurately predicts the solution, demonstrating its efficiency in solving long-time integration of the PDE by respecting temporal causality. Conversely, PINN fails to produce an accurate prediction even at earlier time, e.g., $t = 1$ as shown in Fig. 3(b). The residual loss of PINN at $t = 0$, denoted as $\tilde{\mathcal{L}}_{res}^{PINN}(0; \theta)$, stalls after a certain epoch as displayed in Fig. 4(a), while that of PINN at $t = 1$, $\tilde{\mathcal{L}}_{res}^{PINN}(1; \theta)$, continues to decrease. This suggests that PINN trains the solutions at later time steps even when the solution at earlier time steps are insufficiently trained, violating temporal causality. In contrast, CEEN decreases the residual losses, $\tilde{\mathcal{L}}_{res}^{CEEN}(t; \theta)$, to the tolerance $\epsilon = 1e-4$ at both $t = 0$ and 1. Fig. 4(b) displays the residual losses of PINN and CEEN at each time. Notably, despite PINN having low residual loss across most of the time domain, it fails to predict a solution due to the high loss in the initial time step. Our sequential learning strategy effectively mitigates this issue by reducing the residual loss at each step sequentially from the initial time step.

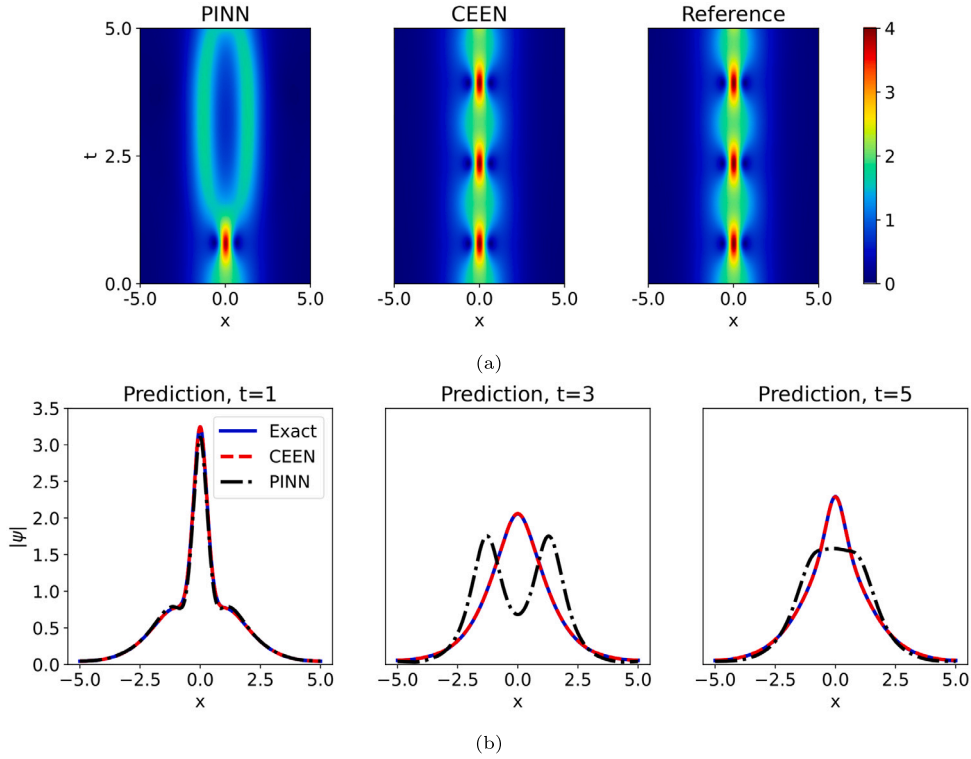


Fig. 5. *Schrödinger equation*: Comparison of the prediction of CEEN and PINN against the reference solution. (a) displays the prediction across the entire spatio-temporal domain. (b) displays it at three different times: $t = 1, 3, 5$. CEEN agrees well with the reference in the entire domain while PINN closely approximates the solution at an earlier time $t = 1$. However, discrepancies between PINN predictions and the reference solution become apparent at later times $t = 3$ and $t = 5$.

4.2. Schrödinger equation

We next consider the Schrödinger equation, which governs a nonlinear wave propagation in quantum mechanical systems. It takes the form:

$$\begin{aligned}
 i\psi_t + 0.5\psi_{xx} + |\psi|^2\psi &= 0, \quad \text{for } t \in [0, 5], x \in [-5, 5], \\
 \psi(0, x) &= 2 \operatorname{sech} x, \\
 \psi(t, -5) &= \psi(t, 5), \\
 \psi_x(t, -5) &= \psi_x(t, 5).
 \end{aligned} \tag{13}$$

In fact, if u and v denote the real and imaginary parts of ψ , respectively, they satisfy the following coupled governing equations.

$$\begin{aligned}
 u_t + 0.5v_{xx} + v(u^2 + v^2) &= 0, \\
 v_t - 0.5u_{xx} - u(u^2 + v^2) &= 0.
 \end{aligned}$$

To apply our method, we discretize the time domain with the step size $h = 1e-2$. The solutions u and v are jointly represented at each time step using a fully connected neural network with two outputs. The architecture of the network includes two hidden layers, each containing 16 neurons. We use $N_x = 501$ equidistant points as sampled points on the spatial domain and the tolerance is $\epsilon = 1e-5$. The chosen values for K and M are 2 and 2, respectively. The setting for PINN method is established in a similar way as outlined in the previous examples.

The results are shown in Fig. 5 where our method shows good agreement with the reference solution across the whole spatio-temporal domain. However, while PINN's predictions shows good agreement up to around $t = 1$, they start to deviate from the reference solution after $t = 1$. Fig. 6(a) shows that $\hat{\mathcal{L}}_r^{\text{PINN}}(1; \theta)$ remains high while $\hat{\mathcal{L}}_r^{\text{PINN}}(2; \theta)$ is sufficiently reduced and remains low during training. This indicates a violation of temporal causality, as the solution at $t = 2$ appears to be settled before the solution at $t = 1$ has been adequately trained. On the other hand, the residual losses of CEEN are sufficiently reduced at both $t = 1$ and 2 to the prescribed threshold $\epsilon = 1e-5$. Fig. 6(b) demonstrates that CEEN achieves a satisfactory reduction of residual losses across the entire time domain while the residual losses of PINN remain noticeably elevated near $t = 1$.

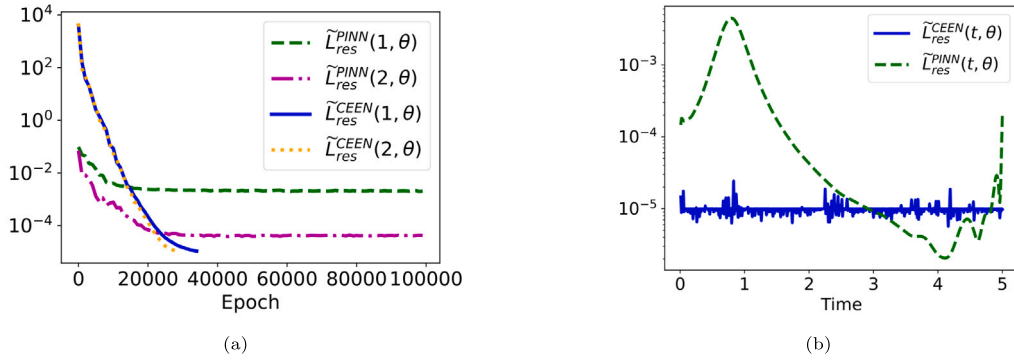


Fig. 6. Residual losses of CEEN and PINN for the Schrödinger equation (13). (a) Epoch vs residual loss. (b) Time vs residual loss.

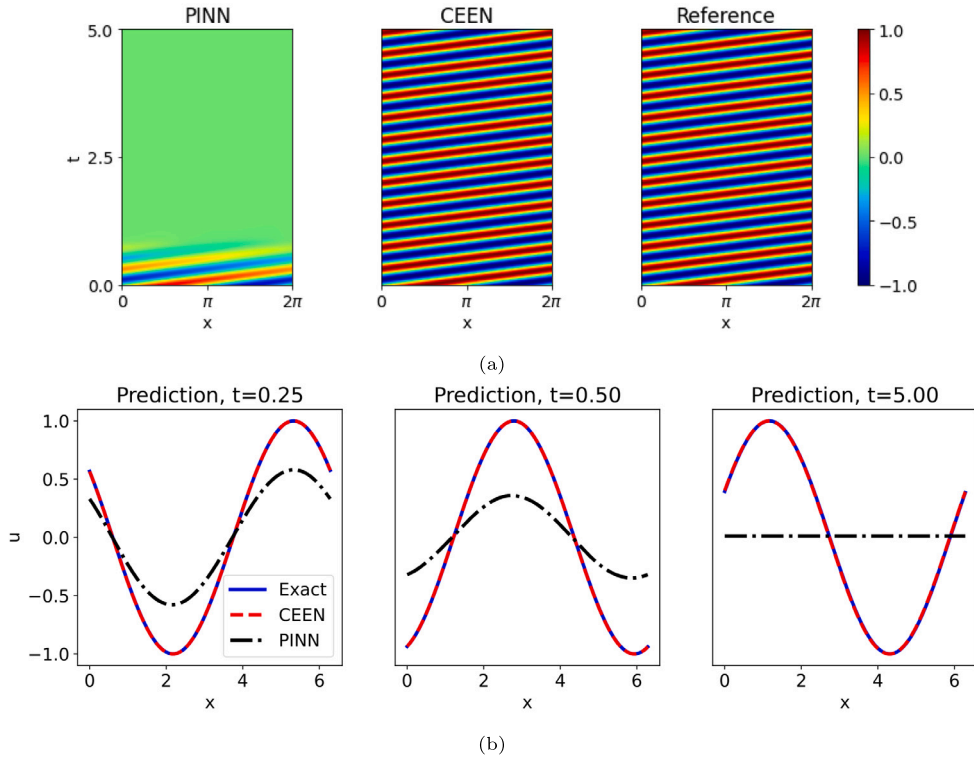


Fig. 7. Convection equation: Comparison of the prediction of CEEN and PINN against the reference solution. (a) displays the prediction across the entire spatio-temporal domain. (b) displays it at three different times: $t = 0.25, 0.5, 5$. CEEN agrees well with the reference in the entire domain while PINN fails to approximate the solution.

4.3. Convection equation

In this section, we consider the convection equation of the form:

$$u_t + 15u_x = 0, \quad \text{for } t \in [0, 5], x \in [0, 2\pi],$$

$$u(0, x) = \sin x,$$

$$u(t, 0) = u(t, 2\pi)$$

$$u_x(t, 0) = u_x(t, 2\pi).$$

(14)

The exact solution for the convection problem can be easily obtained:

$$u(x, t) = \sin(x - 15t).$$

(15)

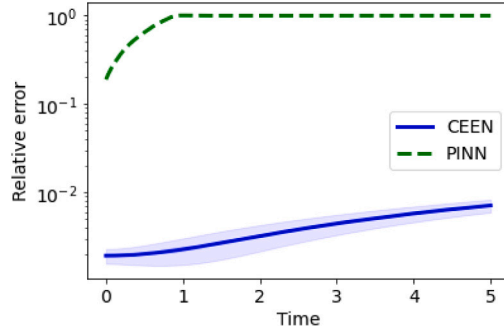


Fig. 8. Relative errors of CEEN and PINN for the convection equation (14). Shaded area is a standard deviation band for the prediction of CEEN obtained by running ten independent simulations.

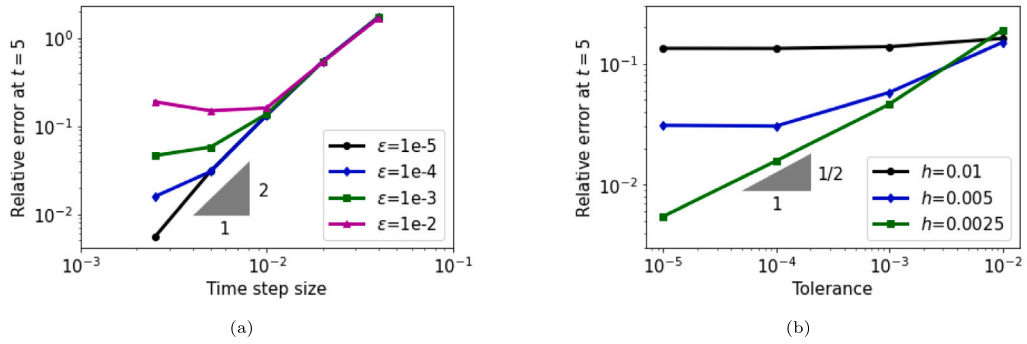


Fig. 9. Relative error of our method at final time ($T = 5$) for different time step sizes h and tolerances ϵ for the convection equation (14). (a) Time step size vs relative error. The second order of accuracy with respect to the time step size h is achieved for sufficiently small tolerance ϵ . (b) Tolerance vs relative error. 1/2 order of accuracy with respect to the tolerance ϵ is achieved for sufficiently small h .

We compare two methods, PINN and CEEN, with the exact solution (15). In CEEN, we discretize the time domain with the time step size $h = 2.5 \times 10^{-3}$. We use $N_x = 32$ equidistant points on the spatial domain Ω as the collocation points and the tolerance is $\epsilon = 1 \times 10^{-5}$. The solution at each time step is represented using a fully-connected neural network. The architecture of the network includes two hidden layers, each containing 15 neurons. The chosen values for K and M used are 1 and 10, respectively. The setting for PINN is established in a similar manner to that described in the previous example.

The results are summarized in Fig. 7, which shows good agreement between our prediction and the reference solution. However, the PINN model falls short, failing to accurately predict the solution even at an earlier time, such as $t = 0.25$, as shown in Fig. 7(b). Fig. 8 illustrates that the relative errors of PINN and FDM are much higher than that of CEEN.

Next, we investigate the relationship between the relative error of the predicted solution at the final time and factors such as the time step size h or the tolerance ϵ . As derived in (10), the error of our method consists of two terms $e_1(\epsilon, h) = |\Omega|^{1/2} \epsilon^{1/2} h$ and $e_2(h) = \frac{1}{12} h^3 \|\mathcal{N}[u]_{tt}(t_*, \cdot)\|_{L^2(\Omega)}$, representing the training loss and the consistency error, respectively. We expect that the second order accuracy with respect to h is attained as $e_1(\epsilon, h)$ has a negligible contribution to the overall error due to a small value of ϵ . Indeed, Fig. 9(a) confirms this, demonstrating a second-order accuracy in time for sufficiently small tolerance ϵ . Alternatively, we expect that the error with respect to ϵ has the order of 1/2, the exponent of ϵ in $e_1(\epsilon, h)$, as $e_2(h)$'s contribution to the error becomes negligible with a sufficiently small h . Indeed, Fig. 9(b) verifies that, for a sufficiently small time step size $h = 0.0025$, our method achieves the error of the order of 1/2.

4.4. Allen-Cahn equation

Our final example deals with the Allen-Cahn equation to highlight the capability of our proposed method in solving phase field models. This equation is commonly used in the area of reaction-diffusion systems which characterizes the phase separation process in multi-component alloy systems. The governing equation for this system can be expressed as follows:

$$\begin{aligned} u_t - 0.0001 u_{xx} + 5u^3 - 5u &= 0, \quad \text{for } t \in [0, 5], x \in [-1, 1] \\ u(0, x) &= x^2 \cos \pi x, \\ u(t, -1) &= u(t, 1), \\ u_x(t, -1) &= u_x(t, 1). \end{aligned} \tag{16}$$

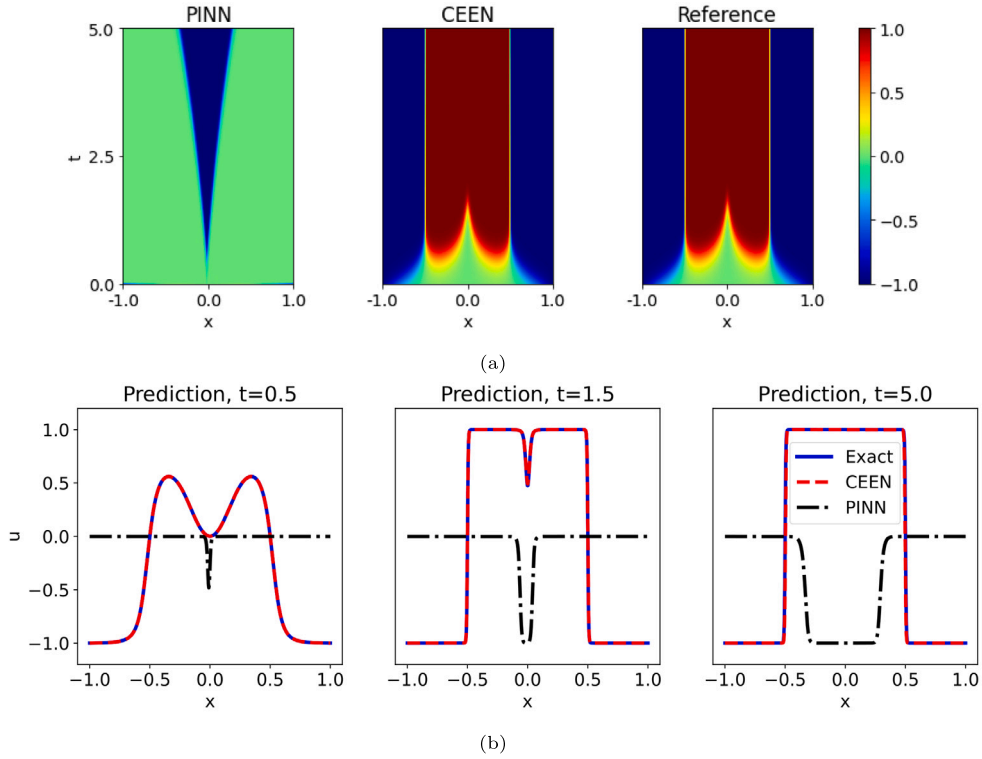


Fig. 10. Allen-Cahn equation: Comparison of the prediction of CEEN and PINN against the reference solution. (a) displays the prediction across the entire spatio-temporal domain. (b) displays it at three different times: $t = 0.5, 1.5, 5$. CEEN agrees well with the reference in the entire domain while PINN fails to approximate the solution.

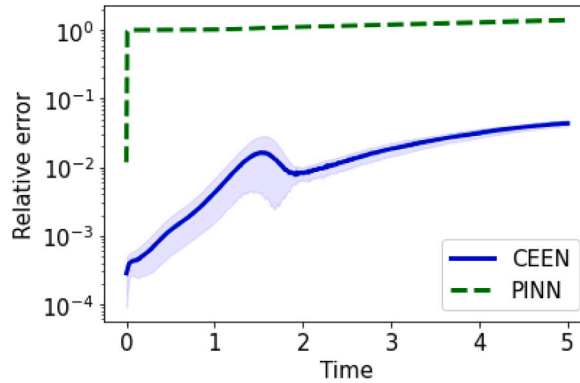


Fig. 11. Relative errors of CEEN and PINN for the Allen-Cahn equation (16). Shaded area is a standard deviation band for the prediction of CEEN obtained by running ten independent simulations..

To implement our method to solve this equation, we discretize the time domain with the step size $h = 1e-2$. A fully-connected neural network with two hidden layers, each containing 32 neurons, is used to represent the solution at each time step. We use $N_x = 201$ equidistant points sampled points and the tolerance is $1e-6$. The chosen values for K and M are 2 and 5, respectively. The setting for PINN method is established in a similar manner to that described in the previous examples.

Fig. 10 shows the comparison of CEEN and PINN against the reference solution. The prediction of our method agrees very well with the reference solution over the entire domain while PINN fails to learn the solution even at earlier time. Our method effectively captures the discontinuity in the solution. Fig. 11 show the relative error of CEEN and PINN over time with CEEN consistently outperforming PINN, thereby underscoring its superior effectiveness.

Table 2
CEEN parameters for the convection equation.

Architecture			Training
NN shape	[2, 15, 15, 1]	Optimizer	Adam
Activation	sin	Learning rate	10^{-3}
M	10	Optimizer parameters	$\beta = (0.999, 0.999)$
K	1	ϵ	10^{-5}
		Maximum iteration	100,000

5. Conclusion

In this study, we have developed and demonstrated a novel sequential learning strategy, termed as CEEN, designed to solve long-time integration of PDEs. We achieved this by dividing the time domain into non-overlapping subintervals, assigning a unique neural network to each subinterval, and constructing a loss function based on the integral form of PDEs within these subintervals. Then, we enforced a sequential training, beginning with the initial time step.

Throughout the application of our method to various PDE problems, including the Korteweg–de Vries (KdV) equation, Schrödinger equation, convection equation, and Allen–Cahn equation, we have consistently observed superior performance compared to the PINN. In particular, our method demonstrated remarkable agreement with reference solutions across entire spatio-temporal domains. Furthermore, the parallel training showed a significant computational speedup, making it an efficient choice for complex problems. We have also demonstrated the second-order accuracy with respect to the time step and the half-order accuracy with respect to the tolerance. These findings validate the theoretical error analysis and provide insights into the accuracy of our method. However, more comprehensive error analysis will be left for future work.

While we have used the trapezoidal rule as one of many potential linear-multistep methods for formulating the loss function, the flexibility of our approach allows for the straightforward incorporation of any other linear-multistep methods, such as Adams–Moulton methods, for loss function construction. This extensibility opens up avenues for further research. High-order methods, for example, are anticipated to accommodate larger time steps and ensure high-order accuracy, which will be investigated in future work.

CRedit authorship contribution statement

Jeahan Jung: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Conceptualization. **Heechang Kim:** Writing – review & editing, Visualization, Software, Methodology, Investigation. **Hyomin Shin:** Writing – review & editing, Visualization, Software, Methodology, Investigation. **Minseok Choi:** Writing – review & editing, Software, Methodology, Investigation, Funding acquisition, Conceptualization, Supervision, Writing – original draft.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

This work was supported by the National Research Foundation of Korea, South Korea [NRF-2021R1C1C1007875].

Appendix A. Comparison between CEEN and classical numerical method

In this Appendix, we conduct a comparison between CEEN and classical numerical methods, focusing on the convection equation discussed in Section 4.3. The presence of a closed-form solution for this equation enables the precise calculation of accuracy. We select the Finite Difference Method (FDM) as the classical numerical method for comparison, using the same time-integrator, the Trapezoidal discretization, as employed in CEEN to ensure a fair comparison. For spatial discretization, a centered difference scheme is utilized. It is noteworthy that the selected FDM is absolute stable [46] and achieves second-order accuracy both in space and time.

For CEEN, we choose the same training configuration as detailed in Section 4.3. The parameters used for this comparison are summarized in Table 2.

Firstly, we fix the number of time steps as $N_t = 2001$ (equivalent to $dt = 0.0025$) and vary the number of spatial grids (N_x) to analyze how the error behaves with respect to N_x . Fig. 12(a) illustrates the relative L_2 error at the final time $t = 5$ for both FDM and CEEN. FDM exhibits second-order accuracy with respect to N_x due to its spatial discretization, achieving an error of 10^{-2} with

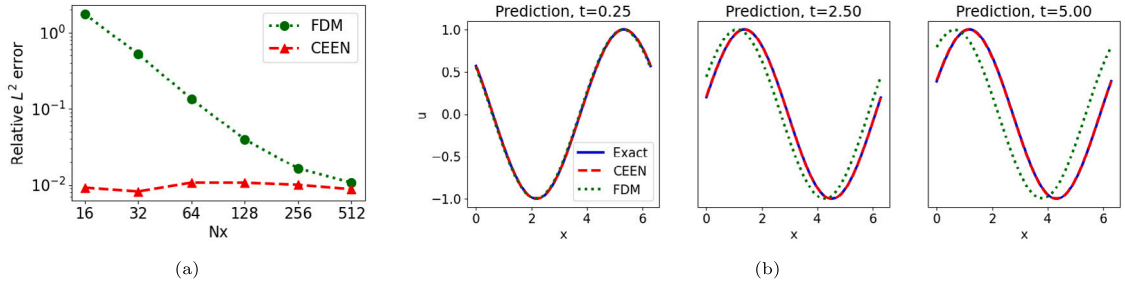


Fig. 12. Comparison of CEEN and finite difference method (FDM). (a) Relative errors at the final time $t = 5$ vs. N_x . (b) Predictions of solution with $N_x = 32$ at three different times.

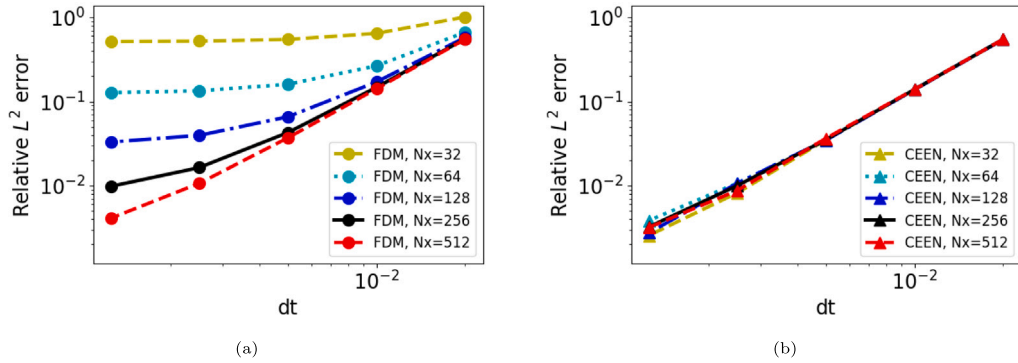


Fig. 13. Relative L_2 errors vs. dt at the final time $t = 5$ for FDM (a) and CEEN (b).

Table 3
CEEN hyperparameters for four PDE problems.

Example	Settings					
	Architecture	Activation	N_x	N_t	ϵ	K
KdV	[5, 16, 16, 1]	tanh	151	501	10^{-4}	3
Schrödinger	[3, 16, 16, 2]	tanh	151	101	10^{-5}	2
Convection	[2, 15, 15, 1]	tanh	101	1001	10^{-5}	1
Allen-Cahn	[1, 32, 32, 1]	tanh	201	501	10^{-5}	2

$N_x = 512$. Conversely, the error of CEEN shows independence of the number of spatial grids, thanks to its precise computation of spatial derivatives through automatic differentiation, as discussed in Section 3.3. Notably, CEEN achieves a lower error with just $N_x = 16$ compared to FDM even with $N_x = 512$. Fig. 12(b) compares the prediction of CEEN and FDM with $N_x = 32$ at three different times: $t = 0.25, 2.5$ and 5 . FDM successfully approximates the solution at the early time $t = 0.25$, but struggles to accurately approximate the solution at later times. However, CEEN predicts the solution accurately at all times.

Secondly, we present a quantitative analysis comparing the relative errors of FDM and CEEN with respect to the time step size dt (equivalent to the number of temporal steps N_t). Fig. 13 illustrates the relative L_2 error at the final time $t = 5$ for both FDM (left panel) and CEEN (right panel). For FDM, when N_x is low, we observe that the error decreases and then plateaus as dt decreases, indicating that the error due to spatial discretization becomes predominant. However, with large number of spatial grids such as $N_x = 512$, the error exhibits second-order accuracy in relation to dt , a direct result of the Trapezoidal rule. This behavior illustrates a well-recognized phenomenon in traditional numerical methods, where the error is influenced by the ratio of dt and dx . Conversely, the error for CEEN consistently achieves second-order accuracy in relation to dt , independent of the number of spatial grids, as discussed in Section 3.3.

Appendix B. Computational cost and accuracy

Here we investigate the computational cost of CEEN compared with that of PINN. Additionally, we explore the effects of parallelization on the computational efficiency and accuracy within the CEEN framework. For consistency and accurate comparison, we employ identical hyperparameter configurations for both CEEN and PINN, as detailed in Section 4, across all four problems under consideration. The specific hyperparameter settings for CEEN are outlined in Table 3.

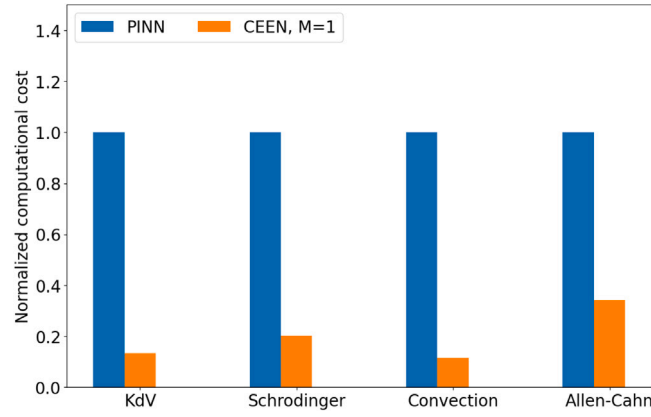


Fig. 14. Comparison of computational cost between PINN and CEEN ($M = 1$). y-axis: the ratio of the computation time of CEEN to that of PINN. For all four problems, the computational cost of PINN is much larger than that of CEEN.

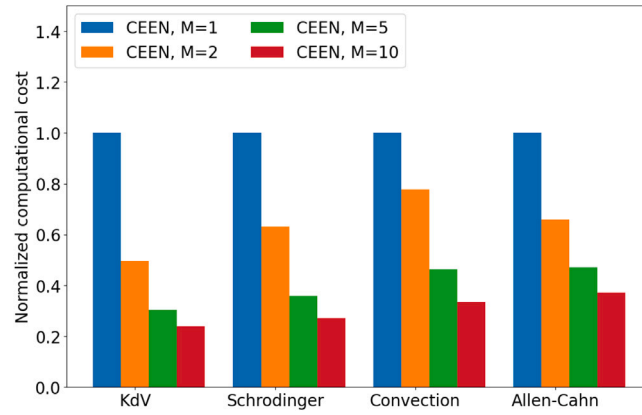


Fig. 15. Comparison of computational cost across different M values in CEEN. y-axis: the ratio of the computation time for the cases in parallel ($M > 1$) to that for the case without parallelization ($M = 1$).

B.1. Computational cost of PINN and CEEN

In this Appendix, we examine the computational efficiency of the CEEN in a non-parallelized setup ($M = 1$) compared to PINN across four case studies presented in Section 4. CEEN involves training multiple neural networks (NNs) sequentially, with each NN designed to model the solution at a given time step. This modular design allows for each NN in the CEEN framework to be more compact, as it needs to accurately capture solution dynamics over a small time interval. On the other hand, PINN employs a single but significantly larger NN to approximate the solution over the entire spatio-temporal domain. Training a smaller NN, as in the CEEN approach, is inherently faster than training a larger one. Even though CEEN incorporates many NNs, dictated by the number of time steps and the total duration of simulation, Fig. 14 demonstrates that CEEN surpasses PINN in computational efficiency for all considered problems. Consequently, CEEN not only reduces computational costs in comparison to PINN but also achieves higher accuracy in long-term simulations of diverse PDE problems, where traditional PINN method fails.

B.2. Effects of parallelization on the computational efficiency and accuracy in CEEN

We investigate the computational efficiency of the CEEN algorithm with parallelization presented in Algorithm 1. We examine the computation times needed to solve each PDE problem in Section 4, utilizing parallelization levels of $M = 2, 5$, and 10, and compare these results to the computation time with $M = 1$. Fig. 15 demonstrates that parallelization reduces computation time. Moreover, it is crucial to highlight that the accuracy of the parallel CEEN, documented in Table 4, remains consistently comparable and does not deteriorate with increasing values of M . This consistency in accuracy underscores the efficacy of the parallel CEEN algorithm, affirming its robustness and effectiveness in computational performance.

Table 4The relative L^2 error at final time for four PDE problems across different values of M .

Example	M			
	1	2	5	10
KdV	2.159e-2	2.433e-2	1.700e-2	2.600e-2
Schrödinger	3.584e-3	3.573e-3	5.080e-3	4.129e-3
Convection	9.232e-3	9.293e-3	9.199e-3	9.204e-3
Allen-Cahn	4.200e-2	3.325e-2	2.229e-2	2.049e-2

References

- [1] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [2] G.E. Karniadakis, I.G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, *Nat. Rev. Phys.* 3 (6) (2021) 422–440.
- [3] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, *SIAM J. Sci. Comput.* 43 (5) (2021) A3055–A3081.
- [4] S. Wang, X. Yu, P. Perdikaris, When and why PINNs fail to train: A neural tangent kernel perspective, *J. Comput. Phys.* 449 (2022) 110768.
- [5] S. Wang, H. Wang, P. Perdikaris, On the eigenvector bias of fourier feature networks: From regression to solving multi-scale pdes with physics-informed neural networks, *Comput. Methods Appl. Mech. Engrg.* 384 (2021) 113938.
- [6] X. Meng, G.E. Karniadakis, A composite neural network that learns from multi-fidelity data: Application to function approximation and inverse PDE problems, *J. Comput. Phys.* 401 (2020) 109020.
- [7] A.D. Jagtap, E. Kharazmi, G.E. Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems, *Comput. Methods Appl. Mech. Engrg.* 365 (2020) 113028.
- [8] L. Yang, X. Meng, G.E. Karniadakis, B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data, *J. Comput. Phys.* 425 (2021) 109913.
- [9] D. Zhang, L. Lu, L. Guo, G.E. Karniadakis, Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems, *J. Comput. Phys.* 397 (2019) 108850.
- [10] L. Yang, D. Zhang, G.E. Karniadakis, Physics-informed generative adversarial networks for stochastic differential equations, *SIAM J. Sci. Comput.* 42 (1) (2020) A292–A317.
- [11] L. Guo, H. Wu, T. Zhou, Normalizing field flows: Solving forward and inverse stochastic differential equations using physics-informed flow models, *J. Comput. Phys.* 461 (2022) 111202.
- [12] J. Jung, H. Shin, M. Choi, Bayesian deep learning framework for uncertainty quantification in stochastic partial differential equations, *SIAM J. Sci. Comput.* 46 (1) (2024) C57–C76.
- [13] W. Zhong, H. Meidani, PI-VAE: Physics-informed variational auto-encoder for stochastic differential equations, *Comput. Methods Appl. Mech. Engrg.* 403 (2023) 115664.
- [14] S. Karumuri, R. Tripathy, I. Bilonis, J. Panchal, Simulator-free solution of high-dimensional stochastic elliptic partial differential equations using deep neural networks, *J. Comput. Phys.* 404 (2020) 109120.
- [15] D. Zhang, L. Guo, G.E. Karniadakis, Learning in modal space: Solving time-dependent stochastic PDEs using physics-informed neural networks, *SIAM J. Sci. Comput.* 42 (2) (2020) A639–A665.
- [16] S. Cho, M. Choi, MGDGAN: Multiple generator and discriminator generative adversarial networks for solving stochastic partial differential equations, *IEEE Access* 10 (2022) 130908–130920.
- [17] H. Shin, M. Choi, Physics-informed variational inference for uncertainty quantification of stochastic differential equations, *J. Comput. Phys.* 487 (2023) 112183.
- [18] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Trans. Neural Netw.* 9 (5) (1998) 987–1000.
- [19] M. Dissanayake, N. Phan-Thien, Neural-network-based approximations for solving partial differential equations, *Commun. Numer. Methods Eng.* 10 (3) (1994) 195–201.
- [20] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G.S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, X. Zheng, TensorFlow: Large-scale machine learning on heterogeneous systems, 2015, Software available from tensorflow.org.
- [21] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: An imperative style, high-performance deep learning library, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, R. Garnett (Eds.), in: *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019, pp. 8024–8035.
- [22] J. Bradbury, R. Frostig, P. Hawkins, M.J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, Q. Zhang, JAX: composable transformations of Python+NumPy programs, 2018.
- [23] R. Matthey, S. Ghosh, A novel sequential method to train physics informed neural networks for Allen Cahn and Cahn Hilliard equations, *Comput. Methods Appl. Mech. Engrg.* 390 (2022) 114474.
- [24] C.L. Wight, J. Zhao, Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks, 2020, arXiv preprint [arXiv:2007.04542](https://arxiv.org/abs/2007.04542).
- [25] B. Wu, O. Hennigh, J. Kautz, S. Choudhry, W. Byeon, Physics informed RNN-DCT networks for time-dependent partial differential equations, 2022, arXiv preprint [arXiv:2202.12358](https://arxiv.org/abs/2202.12358).
- [26] S. Wang, P. Perdikaris, Long-time integration of parametric evolution equations with physics-informed deepONets, *J. Comput. Phys.* (2022) 111855.
- [27] S. Wang, S. Sankaran, P. Perdikaris, Respecting causality is all you need for training physics-informed neural networks, 2022, arXiv preprint [arXiv:2203.07404](https://arxiv.org/abs/2203.07404).
- [28] A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, M.W. Mahoney, Characterizing possible failure modes in physics-informed neural networks, *Adv. Neural Inf. Process. Syst.* 34 (2021) 26548–26560.
- [29] Y. Du, T.A. Zaki, Evolutional deep neural network, *Phys. Rev. E* 104 (4) (2021) 045303.
- [30] H. Zheng, Z. Huang, G. Lin, A physics-constrained neural network for multiphase flows, *Phys. Fluids* 34 (10) (2022) 102102.

- [31] B. Stevens, T. Colonius, FiniteNet: A fully convolutional LSTM network architecture for time-dependent partial differential equations, 2020, arXiv preprint [arXiv:2002.03014](https://arxiv.org/abs/2002.03014).
- [32] P. Ren, C. Rao, Y. Liu, J.-X. Wang, H. Sun, PhyCRNet: Physics-informed convolutional-recurrent network for solving spatiotemporal PDEs, *Comput. Methods Appl. Mech. Engrg.* 389 (2022) 114399.
- [33] Y. Hu, T. Zhao, S. Xu, Z. Xu, L. Lin, Neural-PDE: A RNN based neural network for solving time dependent PDEs, 2020, arXiv preprint [arXiv:2009.03892](https://arxiv.org/abs/2009.03892).
- [34] N. Geneva, N. Zabaras, Modeling the dynamics of PDE systems with physics-constrained deep auto-regressive networks, *J. Comput. Phys.* 403 (2020) 109056.
- [35] J. Qu, W. Cai, Y. Zhao, Learning time-dependent PDEs with a linear and nonlinear separate convolutional neural network, *J. Comput. Phys.* 453 (2022) 110928.
- [36] S. Shit, A.R. Venkatakrishnan, I. Ezhov, J. Lipkova, M. Piraud, B. Menze, Implicit neural solver for time-dependent linear pdes with convergence guarantee, 2019, arXiv preprint [arXiv:1910.03452](https://arxiv.org/abs/1910.03452).
- [37] Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, A. Anandkumar, Physics-informed neural operator for learning partial differential equations, 2021, arXiv preprint [arXiv:2111.03794](https://arxiv.org/abs/2111.03794).
- [38] S. Dong, Z. Li, Local extreme learning machines and domain decomposition for solving linear and nonlinear partial differential equations, *Comput. Methods Appl. Mech. Engrg.* 387 (2021) 114129.
- [39] X. Meng, Z. Li, D. Zhang, G.E. Karniadakis, PPINN: Parareal physics-informed neural network for time-dependent PDEs, *Comput. Methods Appl. Mech. Engrg.* 370 (2020) 113250.
- [40] A.D. Jagtap, K. Kawaguchi, G.E. Karniadakis, Adaptive activation functions accelerate convergence in deep and physics-informed neural networks, *J. Comput. Phys.* 404 (2020) 109136.
- [41] Y.-H. Tseng, T.-S. Lin, W.-F. Hu, M.-C. Lai, A cusp-capturing PINN for elliptic interface problems, *J. Comput. Phys.* 491 (2023) 112359.
- [42] S. Dong, N. Ni, A method for representing periodic functions and enforcing exactly periodic boundary conditions with deep neural networks, *J. Comput. Phys.* 435 (2021) 110242.
- [43] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, *Adv. Neural Inf. Process. Syst.* 33 (2020) 7537–7547.
- [44] D. Zwicker, py-pde: A python package for solving partial differential equations, *J. Open Source Softw.* 5 (48) (2020) 2158.
- [45] C. Ma, L. Wu, E. Weinan, A qualitative study of the dynamic behavior for adaptive gradient algorithms, in: *Mathematical and Scientific Machine Learning*, PMLR, 2022, pp. 671–692.
- [46] D.R. Durran, *Numerical Methods for Fluid Dynamics: With Applications to Geophysics*, vol. 32, Springer Science & Business Media, 2010.